



南开大学软件学院
NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

人工智能导论

讲席教授 张玉志



十四、人工智能中的搜索技术



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

1.什么是搜索技术



- 由于人工智能中有搜索的概念，互联网空间的检索也被称为搜索。两者有关系，又有很大不同，有点乱。我们澄清一下
- Retrieve
 - vt.检索;取回;检索数据;找回;挽回;索回;扭转颓势;
 - n.恢复
- Search
 - n.搜索;检索;查找;搜寻;搜查;
 - v.搜索;查找;搜寻;搜查;搜身;思索，细想(问题答案等)



- 也就是说：retrieve更强调寻找到已知的东西，search更强调探索
- 比如，你问一个同学，昨天忙啥去了？他说去找女朋友去了。有可能两种意思：retrieve女朋友了，是已经有女朋友，去找到这个特定人去见面了。如果是search女朋友去了，就是还没有女朋友，去寻找一个女朋友。



- 传统AI中，搜索技术指的是Search
- 不能确定目标到底是有还是没有
- 状态空间，由规则连在一起（一条规则可以通过去），从一处找到另外一处
 - 最容易想到的例子就是下棋
 - 比如围棋，从空棋盘开始，一人一步，找到没有合理可下的位置
- 从初始位置开始，自然而然的形成一棵树（如果重复节点合在一起，就是一个图）



- AI搜索，就是尝试
- 人们自然而然，经常用到搜索
- 但只有到了在传统人工智能研究时，才仔细地研究了“搜索策略”
- 也就是怎么才能找到解，怎么比较快速的找到解
- 这些技术就是AI中的搜索技术



- 以下四人中只有一人是小偷，只有一人说真话，谁是小偷？
 - 甲：我不是小偷
 - 乙：丁是小偷
 - 丙：乙是小偷
 - 丁：我不是小偷



- 上例中，因为只有一个小偷，甲、丁至少一句是真的，不然两个人都是小偷了。乙丙说的都得是假的。由于乙说丁是小偷，所以甲是小偷。第二句改一下，把丁变成甲，那就是丁是小偷
- 这种方法很简洁，但不通用。
- 中学讲究用技巧找出最快的方法，大学要找出一般的方法
- 中学平面几何，几乎都是这样，解决起来以直觉为主



- 通用的方法，就是假定{甲是小偷，乙是小偷，丙是小偷，丁是小偷}，分别检验，符不符合条件。不管谁是小偷，都算得出来
- 往往会有比搜索更好的方法，但搜索是最通用的方法
- 找出通用的办法，而不是靠技巧，经常就是科学研究的问题



- 搜索成了一种思维方式
- 能解决不少问题
- 比如：如果上面举的例子是100个人，有一个是小偷，每人说一条线索，状态空间大了，大家都是说的真的或者每人一个置信度，想查清楚到底谁没来，可能一般人就迷糊了，但用搜索的办法，没问题，可以很快算出来



- 人工智能中，把搜索策略独立出来研究
- 就是我们现在要学的
- 在数据结构学习中，为了遍历一棵树，我们学过宽度优先、深度优先。也是一种搜索策略
- 启发式 (Heuristic) 是人工智能的搜索策略，基于对未来的
一种评估，得出了很多结论，可以进行数学证明



- 另外一个角度来看：
- 曾经很红的“专家系统”，后来发展为“知识工程”的核心是什么呢？
- 主要就是一些数据
- 加上一些规则（就是知识）[操作这些数据的]
- 加上通用的控制策略
- 这些方法现在依然在用
- 深度学习在没有大量数据可学的时候，无法发挥功效
- 还是要用规则的



- 跟图灵机的定义差不多
- 跟冯诺依曼体系结构的计算机也差不多
- 要了解几个典型的例子，体会出人工智能的味道

专家系统（产生式系统）的基本组成



南开大学软件学院
NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

- 组成三要素：
 - 一个综合数据库——存放信息
 - 一组产生式规则——知识
 - 一个控制系统——规则的解释或执行程序（控制策略）
(推理引擎)

规则的一般形式



- IF <前提> THEN <结论>
- IF <条件> THEN <行动>
- 或者简写为:
 - <前提> - > <结论>
 - <条件> - > <行动>

过程PRODUCTION

- 1, DATA ← 初始数据库
- 2, until DATA满足结束条件, do
- 3, {
- 4, 在规则集中选择一条可应用于DATA的规则R
- 5, DATA ← R应用到DATA得到的结果
- 6, }

➤➤ 一个简单的例子



- 问题：设字符转换规则

$$A \wedge B \rightarrow C$$

$$A \wedge C \rightarrow D$$

$$B \wedge C \rightarrow G$$

$$B \wedge E \rightarrow F$$

$$D \rightarrow E$$

已知：A, B

求：F



一、综合数据库

{x}, 其中x为字符

二、规则集

1, IF $A \wedge B$ THEN C

2, IF $A \wedge C$ THEN D

3, IF $B \wedge C$ THEN G

4, IF $B \wedge E$ THEN F

5, IF D THEN E



三、控制策略
顺序排队

四、初始条件
 $\{A, B\}$

五、结束条件
 $F \in \{x\}$

数据库	可触发规则	被触发规则
A, B	(1)	(1)
A, B, C	(2) (3)	(2)
A, B, C, D	(3) (5)	(3)
A, B, C, D, G	(5)	(5)
A, B, C, D, G, E	(4)	(4)
A, B, C, D, G, E, F		

1, IF $A \wedge B$ THEN C

2, IF $A \wedge C$ THEN D

3, IF $B \wedge C$ THEN G

4, IF $B \wedge E$ THEN F

5, IF D THEN E

传教士与野人过河问题 (M-C问题)

N个传教士，N个野人，一条船，可同时乘坐k个人，要求在任何时刻，在河的两岸，传教士人数不能少于野人的人数。

问：如何过河。

以 $N=3$ ， $k=2$ 为例求解。



初始状态

	L	R
M	3	0
C	3	0
B	1	0



目标状态

	L	R
M	0	3
C	0	3
B	0	1



- 1, 综合数据库
 (M, C, B) ,
 其中: $0 \leq M, C \leq 3, B \in \{0, 1\}$
- 2, 初始状态
 $(3, 3, 1)$
- 3, 目标状态 (结束状态)
 $(0, 0, 0)$



4, 规则集

划过去规则

IF (m, c, 1) THEN (m-1, c, 0)	1传教士
IF (m, c, 1) THEN (m, c-1, 0)	1野人
IF (m, c, 1) THEN (m-1, c-1, 0)	1传教士、1野人
IF (m, c, 1) THEN (m-2, c, 0)	2传教士
IF (m, c, 1) THEN (m, c-2, 0)	2野人

划回来规则

IF (m, c, 0) THEN (m+1, c, 1)	1传教士
IF (m, c, 0) THEN (m, c+1, 1)	1野人
IF (m, c, 0) THEN (m+1, c+1, 1)	1传教士、1野人
IF (m, c, 0) THEN (m+2, c, 1)	2传教士
IF (m, c, 0) THEN (m, c+2, 1)	2野人

- 5, 控制策略：后面研究



用更加复杂的规则

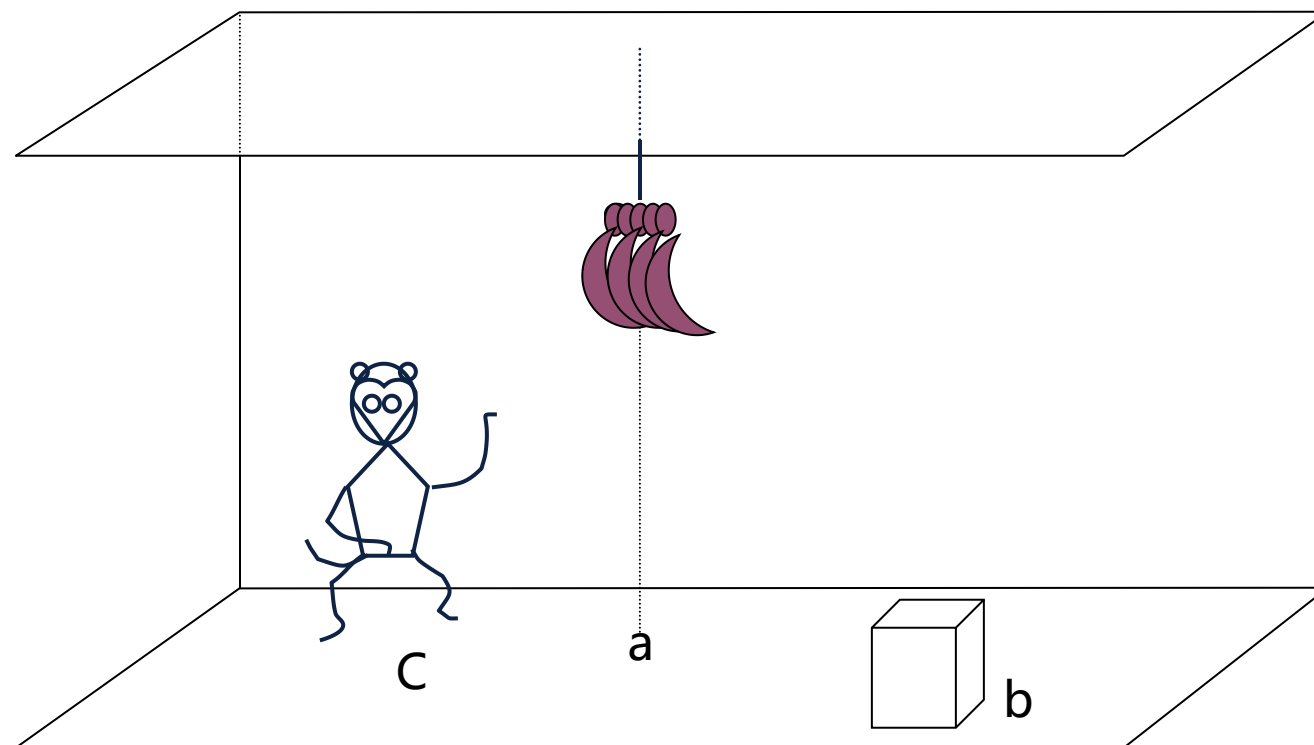
IF (m, c, 1) AND $1 \leq i+j \leq 2$ THEN (m-i, c-j, 0)

IF (m, c, 0) AND $1 \leq i+j \leq 2$ THEN (m+i, c+j, 1)

问题表示例2



- 猴子摘香蕉





1, 综合数据库

(M, B, Box, On, H)

M: 猴子的位置

B: 香蕉的位置

Box: 箱子的位置

On=0: 猴子在地板上

On=1: 猴子在箱子上

H=0: 猴子没有抓到香蕉

H=1: 猴子抓到了香蕉



2, 初始状态

$(c, a, b, 0, 0)$

3, 结束状态

$(x_1, x_2, x_3, x_4, 1)$

其中 $x_1 \sim x_4$ 为变量。



4, 规则集

- r1: IF (x, y, z, 0, 0) THEN (w, y, z, 0, 0) 【猴子可移动】
r2: IF (x, y, x, 0, 0) THEN (z, y, z, 0, 0) 【猴子可以带同位置箱子动】
r3: IF (x, y, x, 0, 0) THEN (x, y, x, 1, 0) 【猴子可以爬上箱子】
r4: IF (x, y, x, 1, 0) THEN (x, y, x, 0, 0) 【猴子可以从箱子下来】
r5: IF (x, x, x, 1, 0) THEN (x, x, x, 1, 1)
【猴子、香蕉、箱子同一位置，且站在箱子上，可以摘香蕉】

其中x, y, z, w为变量

产生式系统的特点



南开大学软件学院
NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

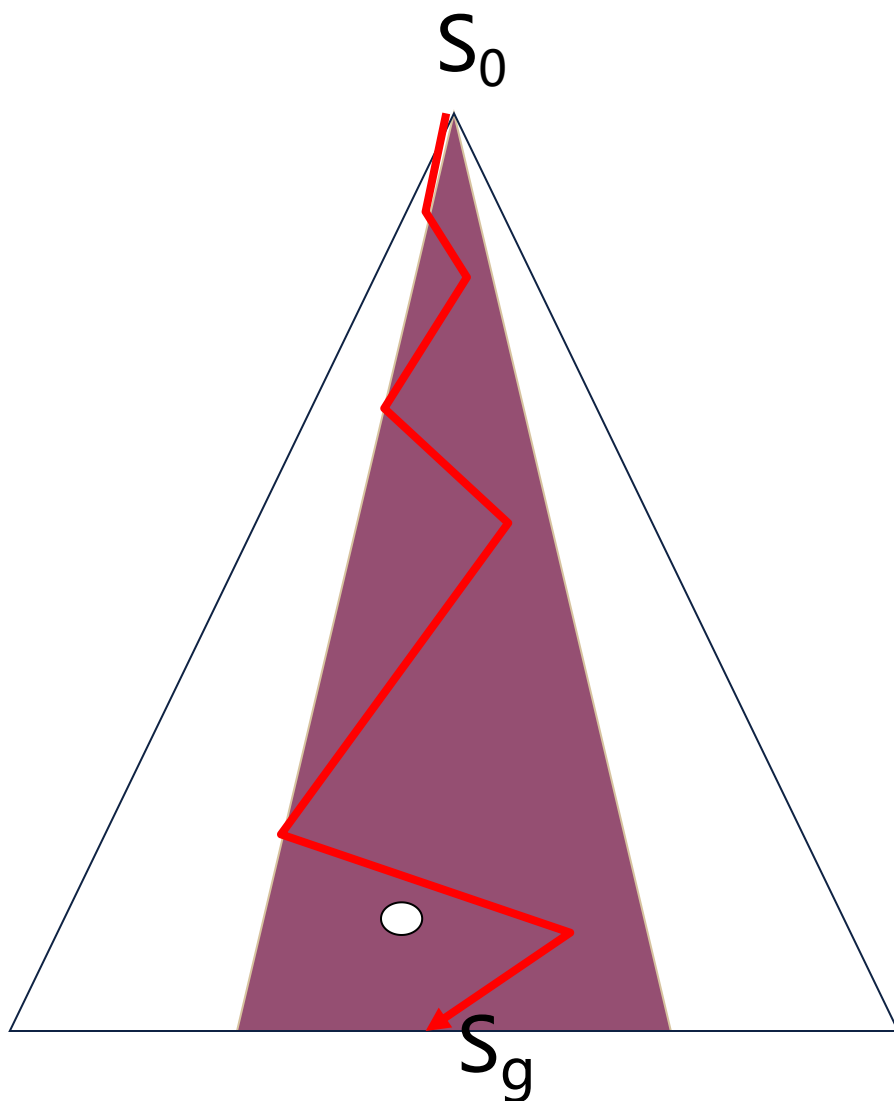
- 数据驱动
- 知识的无序性
- 控制系统与问题无关
- 数据、知识和控制相互独立



- 正向、逆向、双向产生式系统
- 可交换的产生式系统
- 可分解的产生式系统

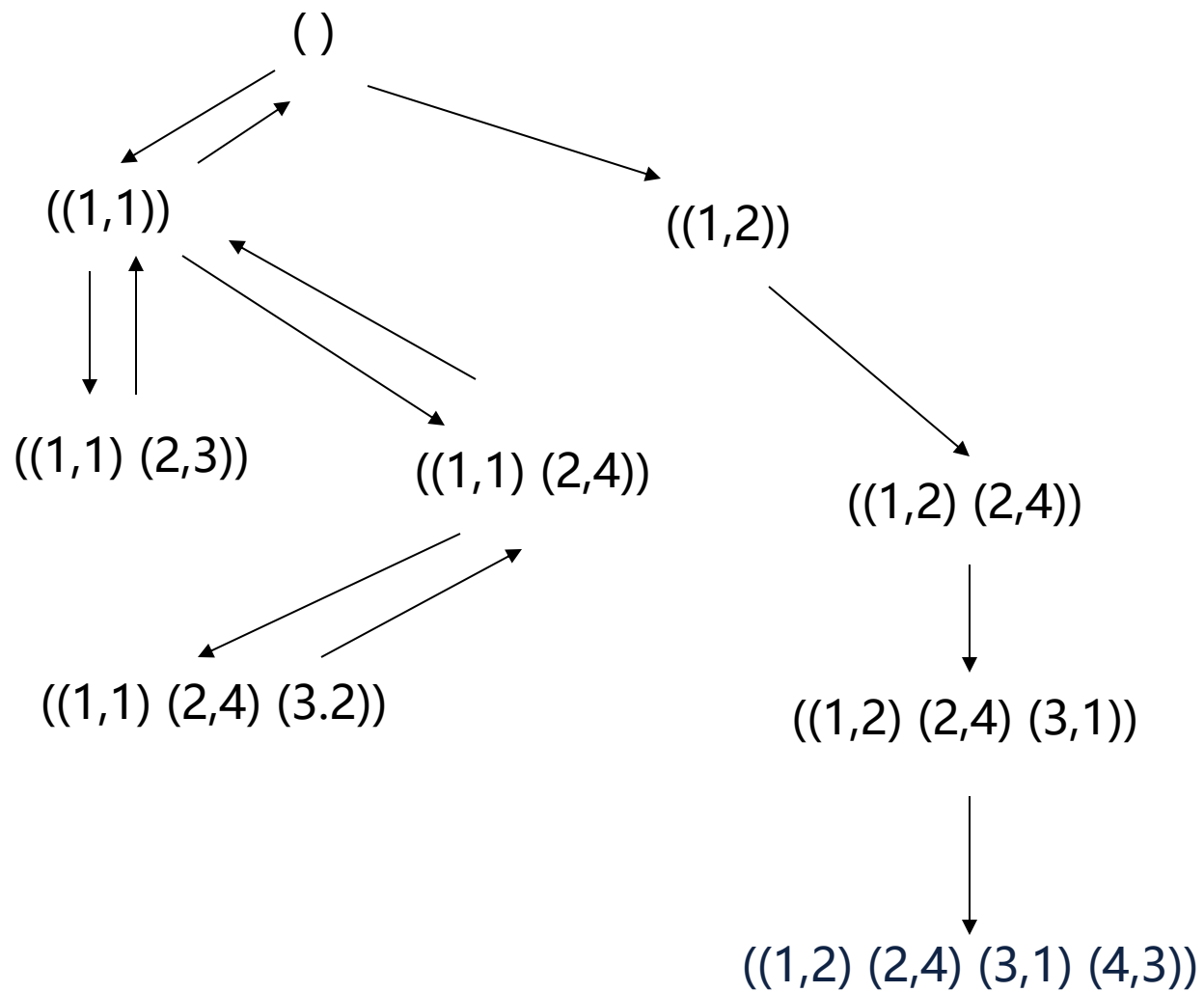
- 内容：
 状态空间的搜索问题。
- 搜索方式：
 盲目搜索
 启发式搜索
- 关键问题：
 如何利用知识，尽可能有效地找到问题的解（最佳解）。

搜索策略:如何从 S_0 出发找到 S_g



- 有哪些常用的搜索算法
- 问题有解时能否找到解
- 找到的解是最佳的吗?
- 什么情况下可以找到最佳解?
- 求解的效率如何

回溯策略



	Q		
			Q
Q			
		Q	



- 回溯策略简单的实现可以用递归算法或者用循环
- 看一下书上的算法就行，在编程课、算法课、数据结构课上应该都用过，我们就不多说了
- 一些简单的搜索，你还是可以用这个方法来处理，特别简单



- 经常遇到的问题和解决办法是：
 - 搜索深度经常很深
 - 解决办法是：控制深度，超过一定深度就算失败了
- 死循环问题（规则应用时循环起来了）
- 解决办法是：记录从初始状态到当前状态的路径，出现循环就认为失败
- 这两个方法扩展一下，就成为比较通用的办法

- 问题的引出

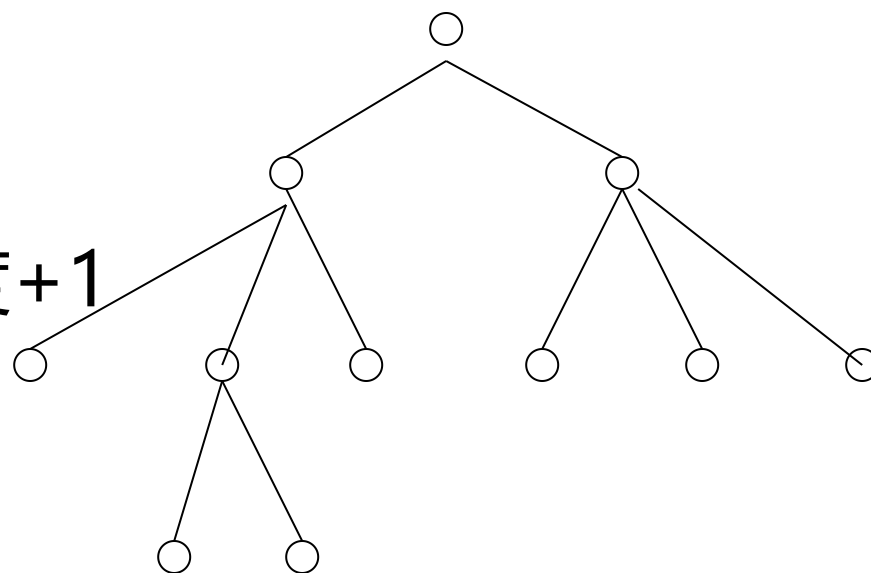
回溯搜索：只保留从初始状态到当前状态的一条路径。

图搜索：保留所有已经搜索过的路径。

- 节点深度：

根节点深度=0

其它节点深度=父节点深度+1



0

1

2

3



- 路径

设一节点序列为 $(n_0, n_1, \dots, n_k)$, 对于 $i=1, \dots, k$, 若节点 n_{i-1} 具有一个后继节点 n_i , 则该序列称为从 n_0 到 n_k 的路径。

- 路径的耗散值

一条路径的耗散值等于连接这条路径各节点间所有耗散值的总和。用 $C(n_i, n_j)$ 表示从 n_i 到 n_j 的路径的耗散值。

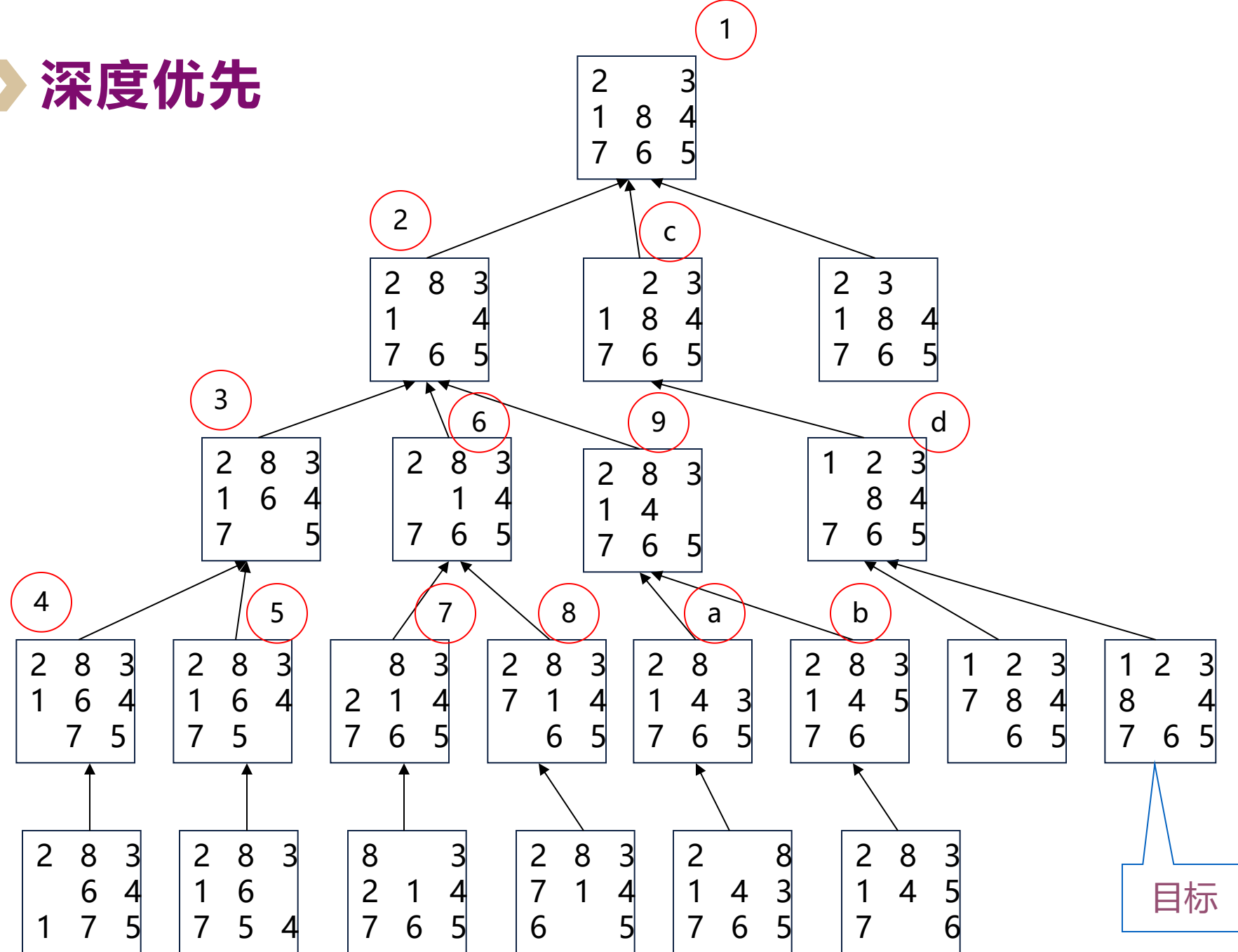
- 扩展一个节点

生成出该节点的所有后继节点, 并给出它们之间的耗散值。这一过程称为“扩展一个节点”。



- 深度优先搜索
- 宽度优先搜索

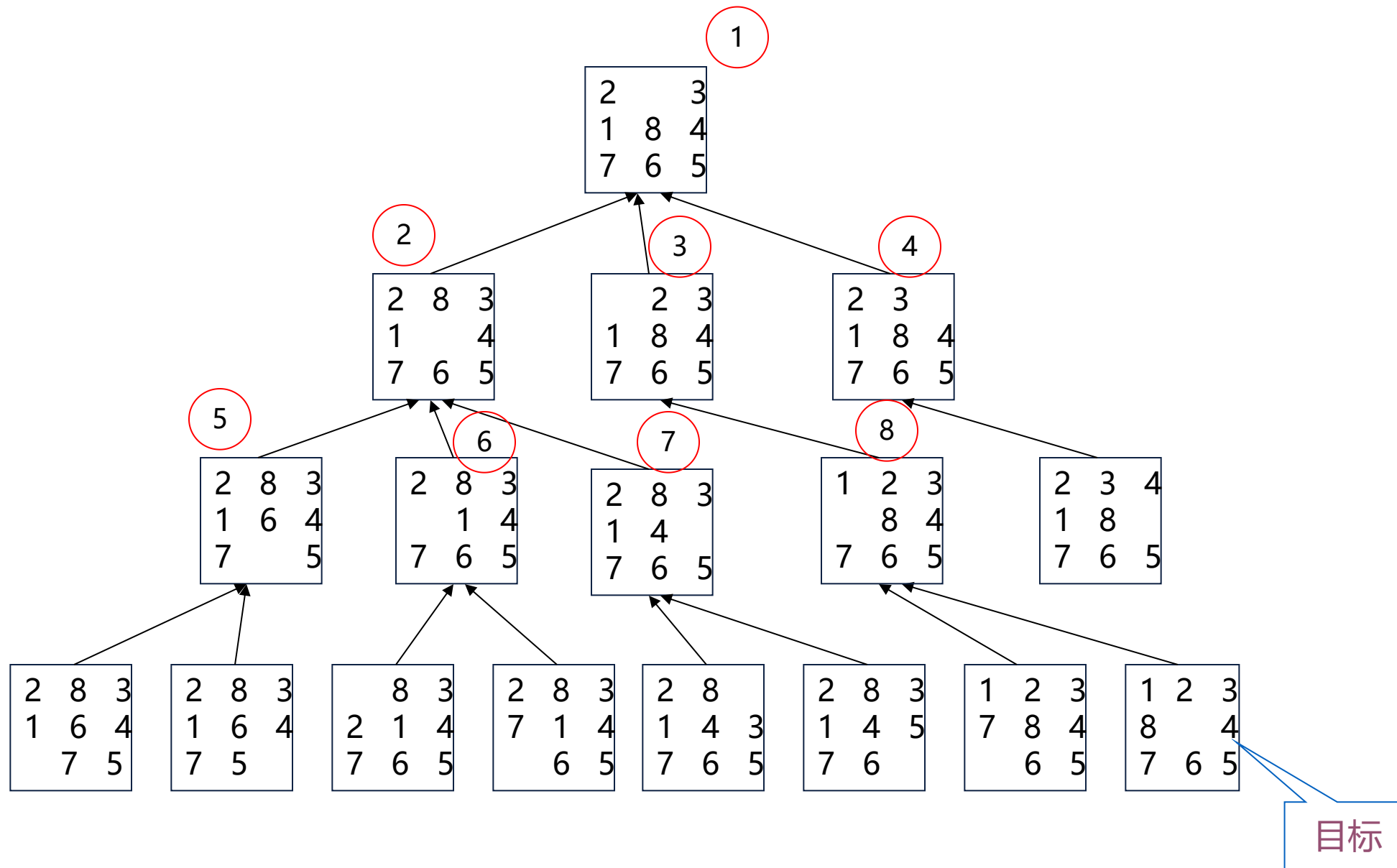
深度优先





- 一般不能保证找到最优解
- 当深度限制不合理时，可能找不到解，可以将算法改为可变深度限制
- 最坏情况时，搜索空间等同于穷举
- 与回溯法的差别：图搜索
- 是一个通用的与问题无关的方法

宽度优先



宽度优先搜索的性质



- 当问题有解时，一定能找到解
- 当问题为单位耗散值，且问题有解时，一定能找到最优解
- 方法与问题无关，具有通用性
- 效率较低
- 属于图搜索方法



- 目的
- 解决宽度优先方法的空间问题和回溯方法不能找到最优解的问题。
- 思想
- 首先给回溯法一个比较小的深度限制，然后逐渐增加深度限制，直到找到解或找遍所有分支为止。

- 利用知识来引导搜索，达到减少搜索范围，降低问题复杂度的目的。
- 启发信息的强度
 - 强：降低搜索工作量，但可能导致找不到最优解
 - 弱：一般导致工作量加大，极限情况下变为盲目搜索，但可能可以找到最优解
- 引入启发知识，在保证找到最佳解的情况下，尽可能减少搜索范围，提高搜索效率。



- 基本思想
- 定义一个评价函数 f ，对当前的搜索状态进行评估，找出一个最有希望的节点来扩展。

1, 启发式搜索算法A (A算法)



- 评价函数的格式:
$$f(n) = g(n) + h(n)$$

$f(n)$: 评价函数
 $h(n)$: 启发函数
- $g^*(n)$: 从s到n的最短路径的耗散值
- $h^*(n)$: 从n到g的最短路径的耗散值
- $f^*(n) = g^*(n) + h^*(n)$: 从s经过n到g的最短路径的耗散值
- $g(n)$ 、 $h(n)$ 、 $f(n)$ 分别是 $g^*(n)$ 、 $h^*(n)$ 、 $f^*(n)$ 的估计值



• A算法

- 1, $OPEN := (s)$, $f(s) := g(s) + h(s)$;
- 2, LOOP: IF $OPEN = ()$ THEN EXIT(FAIL);
- 3, $n := \text{FIRST}(OPEN)$;
- 4, IF $\text{GOAL}(n)$ THEN EXIT(SUCCESS);
- 5, $\text{REMOVE}(n, OPEN)$, $\text{ADD}(n, CLOSED)$;
- 6, $\text{EXPAND}(n) \rightarrow \{m_i\}$,

 计算 $f(n, m_i) := g(n, m_i) + h(m_i)$; $\text{ADD}(m_i, OPEN)$, 标记 m_i 到 n 的指针;

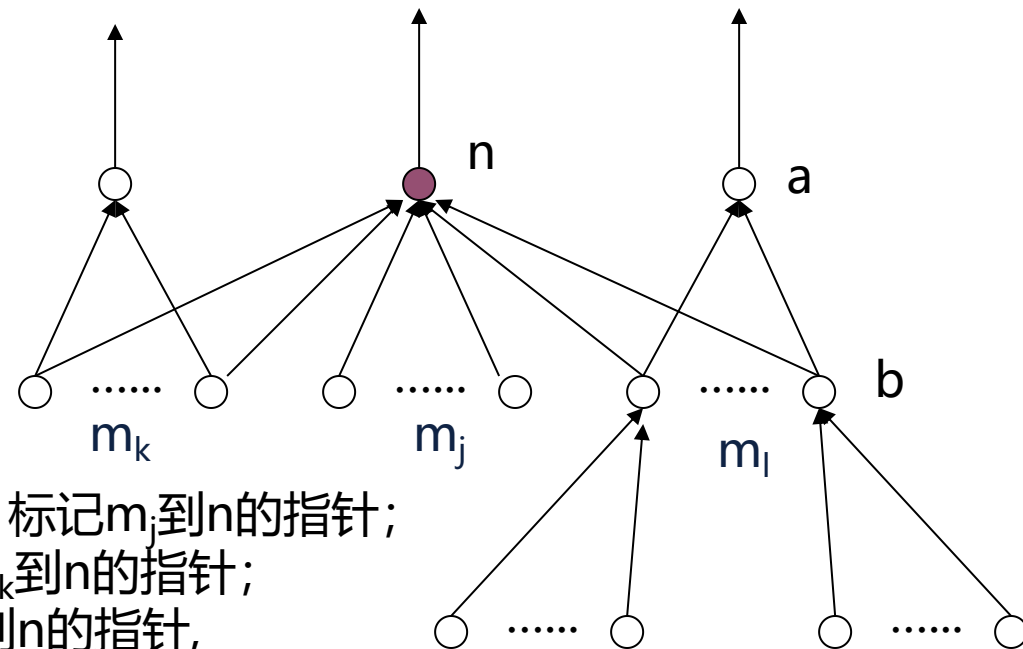
 IF $f(n, m_k) < f(m_k)$ THEN $f(m_k) := f(n, m_k)$, 标记 m_k 到 n 的指针;

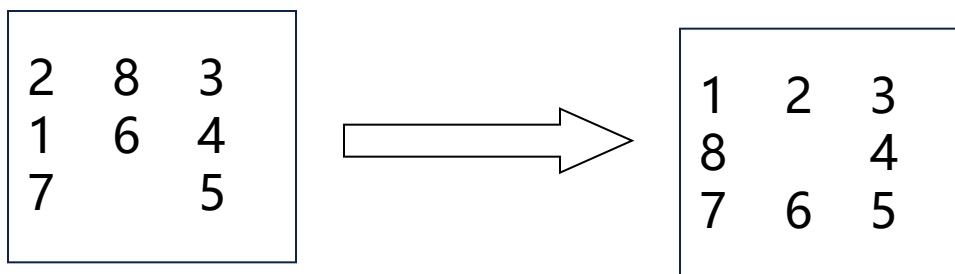
 IF $f(n, m_l) < f(m_l)$ THEN $f(m_l) := f(n, m_l)$, 标记 m_l 到 n 的指针,

$\text{ADD}(m_l, OPEN)$;

- 7, OPEN中的节点按 f 值从小到大排序;

- 8, GO LOOP;



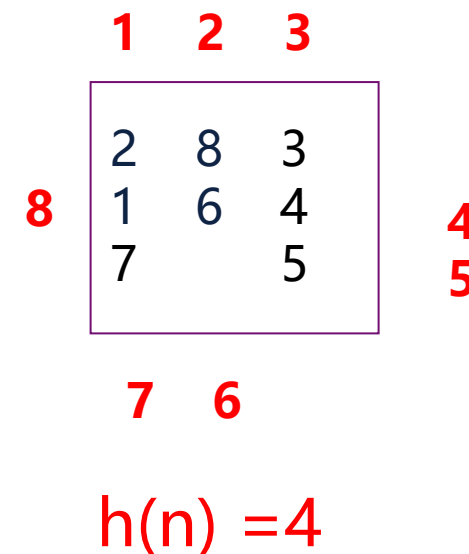


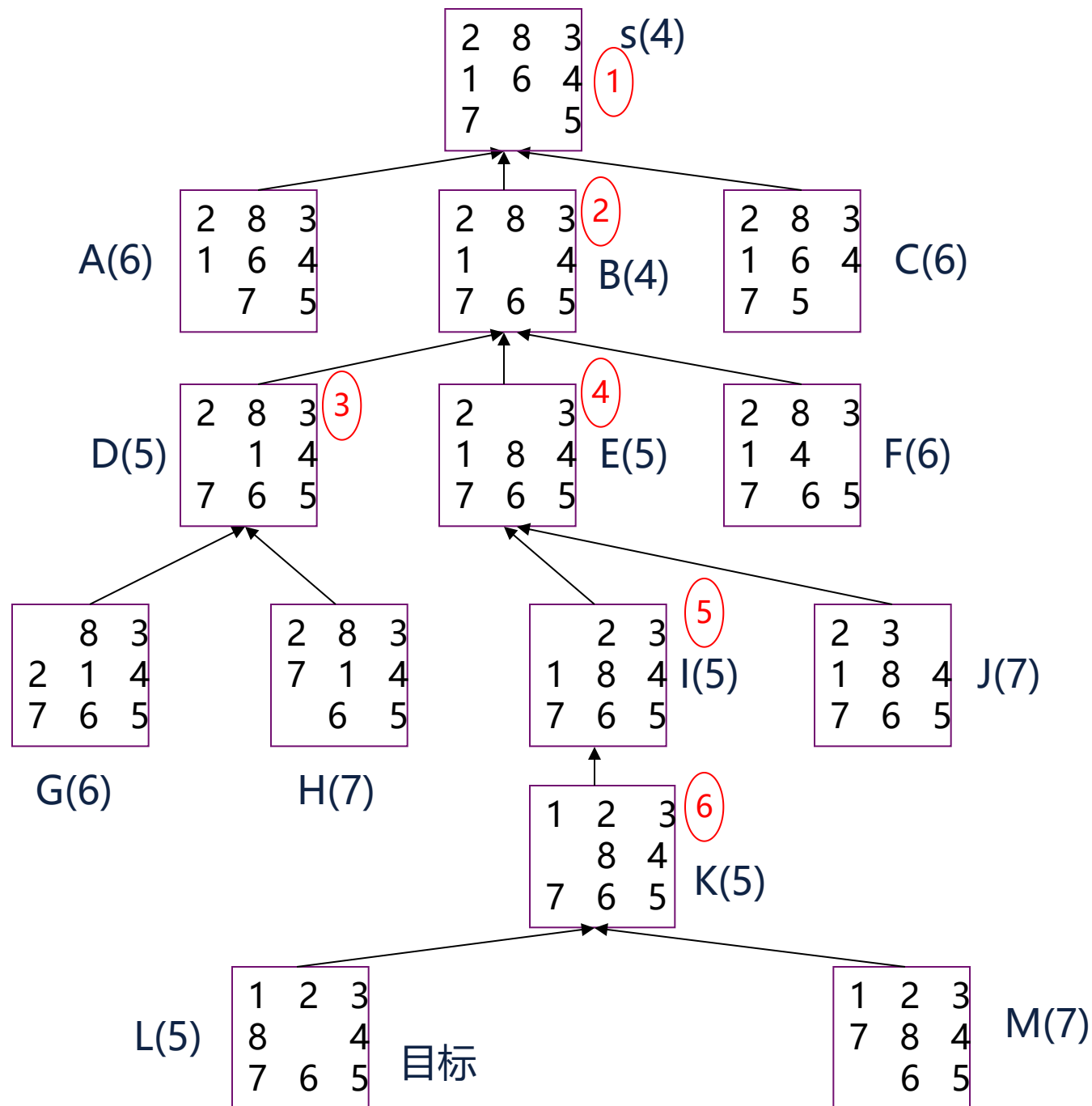
定义评价函数:

$$f(n) = g(n) + h(n)$$

$g(n)$ 为从初始节点到当前节点的耗散值

$h(n)$ 为当前节点“不在位”的将牌数





最佳图搜索算法A* (A*算法)



- 在A算法中，如果满足条件：
$$h(n) \leq h^*(n)$$
则A算法称为A*算法。

A*算法的假设

设 n_i 、 n_j 是任意两个节点，有：

$$C(n_i, n_j) > \varepsilon$$

其中 ε 为大于0的常数

几个等式

$$f^*(s) = f^*(t) = h^*(s) = g^*(t) = f^*(n)$$

其中 s 是初始节点， t 是目标节点， n 是 s 到 t 的最佳路径上的节点。



定理1:

对有限图，如果从初始节点s到目标节点t有路径存在，
则算法A一定成功结束。

引理2.1 :

对无限图，若有从初始节点s到目标节点t的路径，则A*
不结束时，在OPEN表中即使最小的一个f值也将增到任意大，
或有 $f(n) > f^*(s)$ 。

引理2.2:

A*结束前，OPEN表中必存在 $f(n) \leq f^*(s)$ 。



定理2:

对无限图, 若从初始节点 s 到目标节点 t 有路径存在, 则 A^* 一定成功结束。

推论2.1:

OPEN表上任一具有 $f(n) < f^*(s)$ 的节点 n , 最终都将被 A^* 选作扩展的节点。

定理3 (可采纳性定理):

若存在从初始节点 s 到目标节点 t 有路径, 则 A^* 必能找到最佳解结束。



推论3.1:

A*选作扩展的任一节点 n , 有 $f(n) \leq f^*(s)$ 。

定理4: 设对同一个问题定义了两个A*算法 A_1 和 A_2 , 若 A_2 比 A_1 有较多的启发信息, 即对所有非目标节点有 $h_2(n) > h_1(n)$, 则在具有一条从 s 到 t 的路径的隐含图上, 搜索结束时, 由 A_2 所扩展的每一个节点, 也必定由 A_1 所扩展, 即 A_1 扩展的节点数至少和 A_2 一样多。

简写: 如果 $h_2(n) > h_1(n)$ (目标节点除外), 则 A_1 扩展的节点数 $\geq A_2$ 扩展的节点数



- 对h的评价方法

平均分叉树

设共扩展了d层节点，共搜索了N个节点，则：

$$N = \left(1 - b^{*(d+1)}\right) / (1 - b^*)$$

其中， b^* 称为平均分叉树。

b^* 越小，说明h效果越好。

实验表明， b^* 是一个比较稳定的常数，同一问题基本不随问题规模变化。



例：8数码问题，随机产生若干初始状态。

- 使用 h_1 :
 - $d=14, N=539, b^*=1.44;$
 - $d=20, N=7276, b^*=1.47;$
- 使用 h_2 :
 - $d=14, N=113, b^*=1.23;$
 - $d=20, N=676, b^*=1.27$



- 一般来说，A*的算法复杂性是指数型的，可以证明，当且仅当以下条件成立时：

$$\text{abs}(h(n)-h^*(n)) \leq O(\log(h^*(n)))$$

A*的算法复杂性才是非指数型的，但是通常情况下， h 与 h^* 的差别至少是和离目标的距离成正比的。



- 定理5:

若 $h(n)$ 是单调的, 则 A^* 扩展了节点 n 之后, 就已经找到了到达节点 n 的最佳路径。

即: 当 A^* 选 n 扩展时, 有 $g(n)=g^*(n)$ 。

- 定理6:

若 $h(n)$ 是单调的, 则由 A^* 所扩展的节点序列其 f 值是非递减的。即 $f(n_i) \leq f(n_j)$ 。



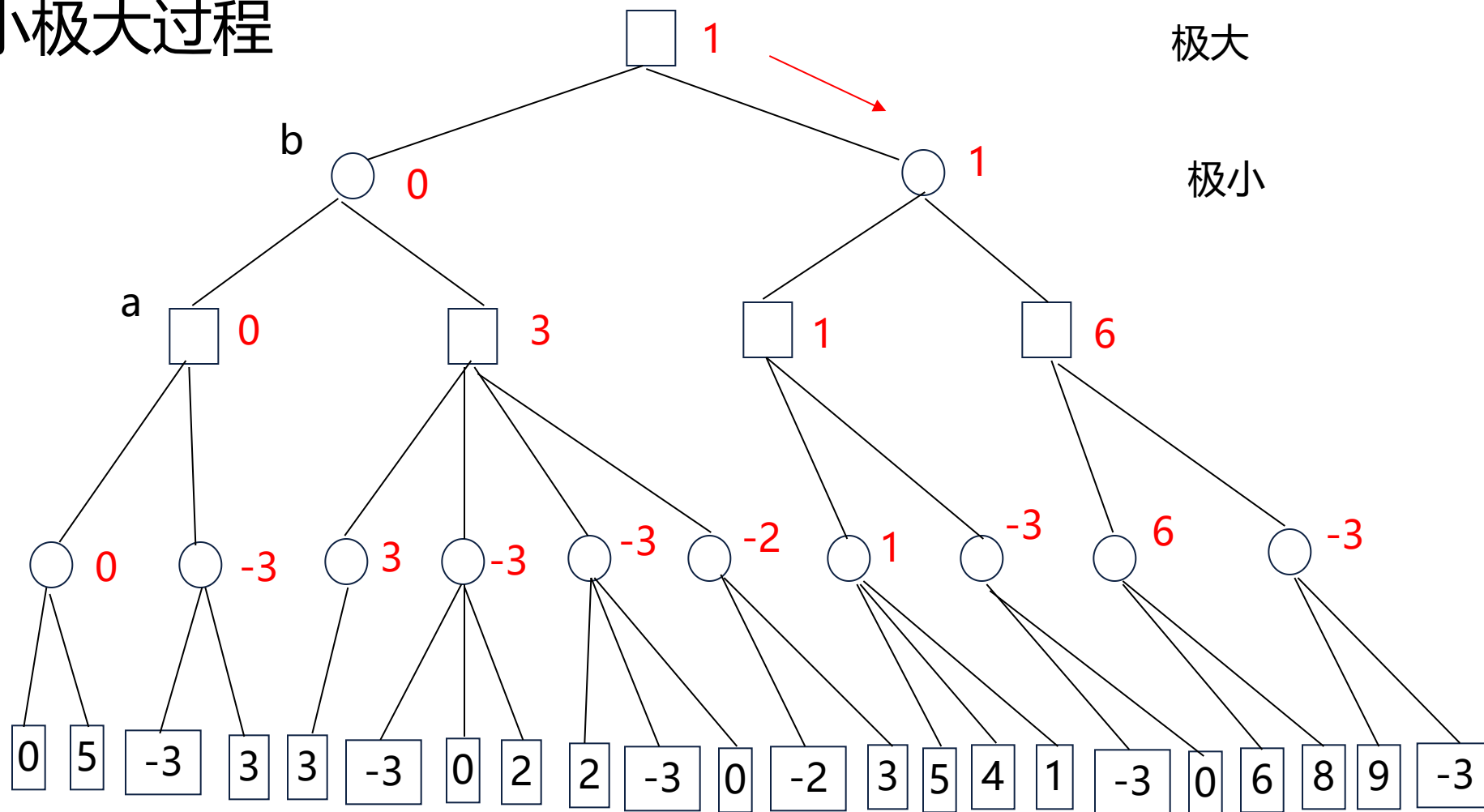
- 博弈问题
 - 双人
 - 一人一步
 - 双方信息完备
 - 零和



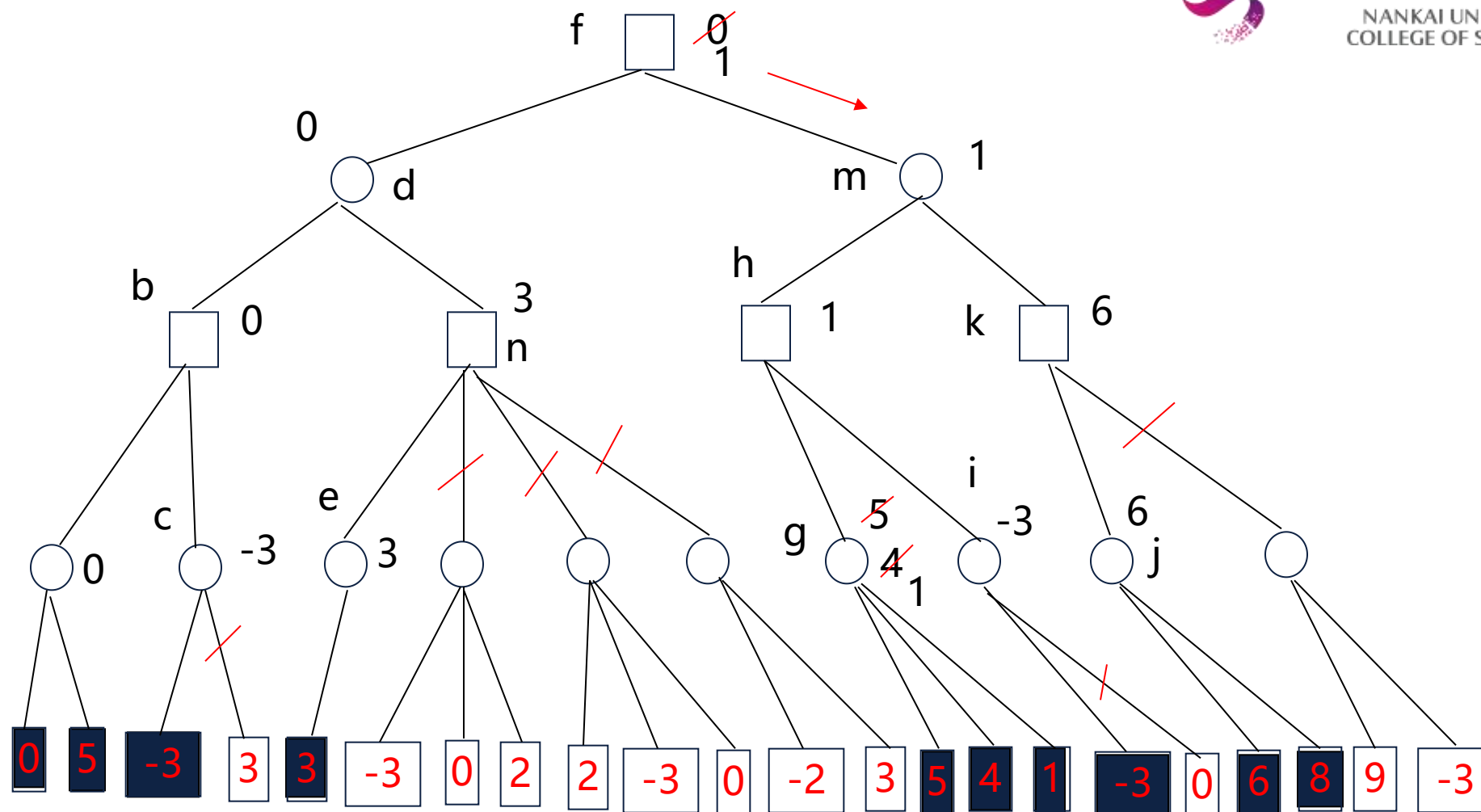
- 中国象棋
 - 一盘棋平均走50步，总状态数约为10的161次方。
 - 假设1毫微秒走一步，约需10的145次方年。
 - 结论：不可能穷举。
- 围棋状态空间更大得多



- 1, 极小极大过程



- 极大节点的下界为 α 。
- 极小节点的上界为 β 。
- 剪枝的条件：
 - 后辈节点的 β 值 $\leq$ 祖先节点的 α 值时, α 剪枝
 - 后辈节点的 α 值 $\geq$ 祖先节点的 β 值时, β 剪枝
- 简记为：
 - 极小 $\leq$ 极大, 剪枝
 - 极大 $\geq$ 极小, 剪枝





- 博弈系统，必然要用到 α - β 搜索
- 包括Alpha Go系统也要用到
- 剪枝是必须要用到的，因为不影响正确性
- 看上去很啰嗦，其实没有啥
- 一个搜索10层左右的下棋程序就有很强的能力了



- 搜索很难不遇到指数复杂性问题
- 剪枝、搜索策略很难改变算法的复杂性
- 怎样才能减小搜索的量?
- 宽度、深度两个方面
- 宽度：从根下面的一层，就去掉大部分的分支，找最有希望的。时间不允许的就不去计算了
- 深度：达到一定的深度就不去计算了，而是给个估算。
Alpha Go 给了一个蒙特卡罗估算



- 搜索是人工智能必须掌握的内容，基本功
- 搜索法让人感觉出了点人工智能的味道
- 搜索方法与人处理问题的方法不太一致
 - 用麻烦（牺牲计算时间）换取一般解法
 - 人是用发现一些规律的方法
 - 围棋的口诀（如：立二拆三）
 - 目的是一致的
- 是现代科学思维方式的代表



- 在虚拟空间里搜索，也是理想实验的一个过程
- 理想实验是一个非常重要的科研基本素质
- 理想实验都不行，就不要去做了，防止无谓的想象



- 人的一生，就是一个启发式搜索的过程
- 每个人的目标是不一致的
- 要达到你的目标，需要制定一个启发式函数
- 你的启发式函数越精确，越容易成功
- 有的人偏宽度优先，喜欢各种不同方向的尝试
- 有的人偏深度优先，找到一条路就走下去

- 都对，都不完善



- 做一件事、一个公司的发展也都是一个搜索的过程
- 比如，改造一个程序
- 比如，公司的多元化经营、还是专业化发展
- 比如，决策的民主化、集思广益，还是聪明者决策



- 我们的人工智能导论课，哪怕你只学会了搜索，也是很有用的
- 所以，认真完成搜索的实验
- 真正的体会一下，不同的搜索策略，会产生什么影响



- 动手编程，实现一个搜索
- 8数码（15数码问题）
- 宽度优先搜索
- 深度优先搜索
- 启发式搜索，给出至少两个启发式函数
- 随机决策选择



- 显示搜索了多少节点，搜索了多少时间
- 可以显示内存占用的情况
- 基本摸清楚，不同的策略的差异
- 先从8数码做起，考虑要扩展到15数码，程序可扩展性易改造的问题
- 找出八数码问题最长的一个搜索链
- 水平较好的同学，程序可以动态演示搜索的过程
- 我们会找一节课，让同学们上台讲解。加分的！



谢谢！ 下课



十五、归结原理



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE



- 我们的课程强调了关联、类比
- 这个思维方式经常是很重要的
- 真正新的知识和思维方式，其实人的一生学习并不是很多
- 总有一些像的地方，我们才可以用经验判断
- 一个新知识出来没多长时间，总有人就开始讲了，怎么能讲出那么多的东西来？
- 类比加想象！



- 年轻时，多数人对历史不感兴趣
- 逐步的，对历史很多人开始有兴趣了
- “历史”的作用在于推断未来
- 别人发生的事可以用来推断自己遇到的
- 这种推断不一定都对，但很有意义
- 假如你将来工作、创业了，遇到了一些问题，怎么解决？
 - 大学的经历，社会经验，多数问题就能解决了



- 作为未来的人才，希望你们注重这样几个问题：
- 为人处世：
 - 不要太算计，不要利益最大化
 - “允公允能”，发扬风格，做善事。能战胜80%的人
 - “袁黄，袁了凡”的故事
- 要学会处理事情：
 - 闭环、有“返回值”、目标（目的，时间）
- 描述事情：
 - 注意要讨论有共同定义的事

早期人工智能的一个重要成果：归结原理



- 几乎任何一本人工智能教材，都要讲“归结原理”
 - 我们也需要讲一下，主要是要靠你自己认真读一个晚上
 - 不适合课堂上讲。特点是：很繁琐，但不难！
 - 我们只讲一下大体思路。考试时一定要考的。
-
- 搜索、归结原理是传统人工智能的主要成果的核心
-
- 从归结原理发展出来了Prolog语言，当年日本“五代机”的机器语言。五代机的失败，是第二次人工智能高潮衰落的标志



- 归结原理是一种定理证明方法，1965年由Robinson提出，从理论上解决了定理证明问题。归结原理的提出，对机器定理证明问题起到了巨大的推动作用。
- 我们回顾一下我们解线性方程的知识
 - 多元一次方程组
 - 跟我们今天讲的归结原理太像了



- 是一种形式语言，具有严密的理论体系
- 是一种常用的知识表示方法
 - 例：
 - City (北京)
 - City (上海)
 - Age (张三, 23)
 - $(\forall x)(\forall y)(\forall z)(F(x, y) \wedge F(y, z) \rightarrow GF(x, z))$

- 归结原理就是对子句集进行一种操作：规约
- 无量词约束
- 元素只是文字的析取
- 否定符只作用于单个文字
- 元素间默认为合取
- 例： $\{\sim I(z) \vee R(z), I(A), \sim R(x) \vee L(x), \sim D(y)\}$
- 就像多元一次方程组的一个个方程

化子句集的方法



例: $(\exists z) (\forall x)(\exists y)\{[(P(x) \vee Q(x)) \rightarrow R(y)] \vee U(z)\}$

1, 消蕴涵符

理论根据: $a \rightarrow b \Rightarrow \sim a \vee b$

$(\exists z) (\forall x)(\exists y)\{[\sim(P(x) \vee Q(x)) \vee R(y)] \vee U(z)\}$

2, 移动否定符

理论根据: $\sim(a \vee b) \Rightarrow \sim a \wedge \sim b$

$\sim(a \wedge b) \Rightarrow \sim a \vee \sim b$

$\sim(\exists x)P(x) \Rightarrow (\forall x)\sim P(x)$

【南开: 不存在腐败, 任意一个都不腐败】

$\sim(\forall x)P(x) \Rightarrow (\exists x)\sim P(x)$

【天津: 并非所有人都腐败, 存在一个不腐败的】

$(\exists z) (\forall x)(\exists y)\{[(\sim P(x) \wedge \sim Q(x)) \vee R(y)] \vee U(z)\}$

3, 变量标准化

即：对于不同的约束，对应于不同的变量

$$(\exists x)A(x) \vee (\exists x)B(x) \Rightarrow (\exists x)A(x) \vee (\exists y)B(y)$$

4, 量词左移

$$(\exists x)A(x) \vee (\exists y)B(y) \Rightarrow (\exists x) (\exists y) \{A(x) \vee B(y)\}$$

5, 消存在量词 (skolem化)

原则：对于一个受存在量词约束的变量，如果他不受全程量词约束，则该变量用一个常量代替，如果他受全程量词约束，则该变量用一个函数代替。

$$\begin{aligned} & (\exists z) (\forall x)(\exists y)\{[(\sim P(x) \wedge \sim Q(x)) \vee R(y)] \vee U(z)\} \\ \Rightarrow & (\forall x) \{[(\sim P(x) \wedge \sim Q(x)) \vee R(f(x))] \vee U(a)\} \end{aligned}$$

化子句集的方法（续2）



6, 化为合取范式

即 $(a \vee b) \wedge (c \vee d) \wedge (e \vee f)$ 的形式

$$(\forall x)\{[(\sim P(x) \wedge \sim Q(x)) \vee R(f(x))]\vee U(a)\}$$

$$\Rightarrow (\forall x)\{(\sim P(x) \wedge \sim Q(x)) \vee R(f(x))\vee U(a)\}$$

$$\Rightarrow (\forall x)\{[\sim P(x) \vee R(f(x))\vee U(a)] \wedge$$
$$[\sim Q(x))\vee R(f(x))\vee U(a)]\}$$

7, 隐去全程量词

$$\{[\sim P(x) \vee R(f(x))\vee U(a)] \wedge [\sim Q(x))\vee R(f(x))\vee U(a)]\}$$

化子句集的方法（续3）



8, 表示为子句集

$$\{\sim P(x) \vee R(f(x)) \vee U(a), \sim Q(x) \vee R(f(x)) \vee U(a)\}$$

9, 变量标准化（变量换名）

$$\{\sim P(x_1) \vee R(f(x_1)) \vee U(a), \sim Q(x_2) \vee R(f(x_2)) \vee U(a)\}$$

- 只有六种符号： $\forall$, $\exists$; $\rightarrow$, $\vee$, $\wedge$, $\sim$
- 简化成子句集
- 1.消掉蕴含
- 2.把否定的都压倒最里面去
- 3. $\forall\exists$, 无关的变量取不同名字, 从而统统移到最左边
- 4.去掉存在量词, 无约束变常量, 有约束变函数
- 5.把所有的全程量词都去掉
- 6. 第一层都变成 “与” 关系, 一个个子句
 - 变成了 “与”, 就拆出来
- 第二层都变成 “或” 关系
- 非常像代数式的变换
$$P \vee (Q \wedge R) \Rightarrow (P \vee Q) \wedge (P \vee R)$$

定理：

若 S 是合式公式 F 的子句集，则 F 永假的充要条件是 S 不可满足。

S 不可满足：若 $\text{nil} \in S$ ，则 S 不可满足。

- 目标的否定连同已知条件一起，化为子句集，并给出一种变换的方法，使得 $S \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$ 同时保证当 S_n 不可满足时，有 S 不可满足。
- 实际上这就是反证法的应用

4.2 归结方法（命题逻辑）



- 设子句： $C_1 = L \vee C_1'$, $C_2 = (\sim L) \vee C_2'$
则归结式C为： $C = C_1' \vee C_2'$

- 定理：
子句集 $S = \{C_1, C_2, \dots, C_n\}$ 与子句集 $S_1 = \{C, C_1, C_2, \dots, C_n\}$ 的不可满足性是等价的。其中，C是 C_1 和 C_2 的归结式。

设公理集:

$P,$

$(P \wedge Q) \rightarrow R,$

$(S \vee T) \rightarrow Q,$

T

求证: R

- 化子句集:

- $(P \wedge Q) \rightarrow R$

- $\Rightarrow \sim(P \wedge Q) \vee R$

- $\Rightarrow \sim P \vee \sim Q \vee R$

- $(S \vee T) \rightarrow Q$

- $\Rightarrow \sim(S \vee T) \vee Q$

- $\Rightarrow (\sim S \wedge \sim T) \vee Q$

- $\Rightarrow (\sim S \vee Q) \wedge (\sim T \vee Q)$

- $\Rightarrow \{\sim S \vee Q, \sim T \vee Q\}$

子句集:

(1) P

(2) $\sim P \vee \sim Q \vee R$

(3) $\sim S \vee Q$

(4) $\sim T \vee Q$

(5) T

(6) $\sim R$ (目标求反)

归结:

(7) $\sim P \vee \sim Q$ (2, 6)

(8) $\sim Q$ (1, 7)

(9) $\sim T$ (4, 8)

(10) nil (5, 9)



- 怎么理解、记住?
- $L \vee C_1'$, $(\sim L) \vee C_2'$ 归结式 $C_1' \vee C_2'$
- L有可能真, 也可能假
- L为假时 C_1' 必为真;
- L为真时, $(\sim L)$ 必为假, C_2' 必为真
- 不管怎么样: $C_1' \vee C_2'$ 总能成立
- 但这样一变化, L没了。消掉了L



- 为什么总能证明出来?
- 如果各个式子都出现 L ，不出现 $\sim L$ ，怎么办?
- 或者各个式子都出现 $\sim L$ ，不出现 L ，怎么办?
- 各个谓词都出现这种情况怎么能证明呢?
- 实际上不会出现的。想想，如果出现的都是 L ，令 L 为永真（相反的情况令 L 为永假）。各个式子都如此，各个子句都为真了。本来就推不出NIL

4.3 谓词逻辑的归结原理



- 问题：如何找归结对

例： $P(x) \vee Q(y), \sim P(f(y)) \vee R(y)$

$P(A) \vee Q(y), \sim P(f(y)) \vee R(y)$

- 基本概念

- 置换

$s = \{t_1/v_1, t_2/v_2, \dots, t_n/v_n\}$

对公式E实施置换s后得到的公式称为E的例，记作 E_s 。

例： $s1 = \{z/x, A/y\}$, 则：

$P[x, f(y), B]_s = P[z, f(A), B]$



- 合一

如果存在一个S置换, 使得 $\{E_i\}$ 中 $E_{1s}=E_{2s}=E_{3s}=\dots=E_{ns}$, 则称 $\{E_i\}$ 是可合一的。S为 $\{E_i\}$ 的合一者。

例: $\{P(x, f(y), B), P(z, f(B), B)\}$

置换 $s=\{A/x, B/y, A/z\}$ 是一个合一者, 因为:

$$P(x, f(y), B)_s = P(A, f(B), B)$$

$$P(z, f(B), B)_s = P(A, f(B), B)$$

置换 $s=\{z/x, B/y\}$ 和置换 $s=\{x/z, B/y\}$ 也都是这两个谓词公式的合一者。

结论: 合一者不唯一。



- 最一般合一者 (mgu)
置换最少, 限制最少, 产生的例最具一般性。

如前面的例子:

$\{P(x, f(y), B), P(z, f(B), B)\}$

对于置换 $\{A/x, B/y, A/z\}$, 产生的例是: $P(A, f(B), B)$

对于置换 $\{z/x, B/y\}$, 产生的例是: $P(z, f(B), B)$

- mgu也不是唯一的。

例: $\{P(x, x, z), P(f(y), f(B), y)\}$

前缀表示:

$(P \ x \ x \ z)$

$(P \ (f \ y) \ (f \ B) \ y)$

置换: $\{(f \ y)/x\}$

$(P \ (f \ y) \ (f \ y) \ z)$

$(P \ (f \ y) \ (f \ B) \ y)$

置换: $\{B/y\}$, 并使得 $\{(f \ B)/x\}$

$(P \ (f \ B) \ (f \ B) \ z)$

$(P \ (f \ B) \ (f \ B) \ B)$

置换: $\{B/z\}$

得到置换: $\{(f \ B)/x, B/y, B/z\}$

置换后的结果: $(P \ (f \ B) \ (f \ B) \ B)$



- 对于子句 $C_1 \vee L_1$ 和 $C_2 \vee L_2$, 如果 L_1 与 $\sim L_2$ 可合一, 且 s 是其合一者, 则 $(C_1 \vee C_2)_s$ 是其归结式。
- 例: $P(x) \vee Q(y), \sim P(f(z)) \vee R(z)$
 $\Rightarrow Q(y) \vee R(z)$

设公理集:

$$(\forall x)(R(x) \rightarrow L(x))$$

$$(\forall x)(D(x) \rightarrow \sim L(x))$$

$$(\exists x)(D(x) \wedge I(x))$$

求证: $(\exists x)(I(x) \wedge \sim R(x))$

化子句集:

$$(\forall x)(R(x) \rightarrow L(x))$$

$$\Rightarrow (\forall x)(\sim R(x) \vee L(x))$$

$$\Rightarrow \sim R(x) \vee L(x) \quad (1)$$

$$(\forall x)(D(x) \rightarrow \sim L(x))$$

$$\Rightarrow (\forall x)(\sim D(x) \vee \sim L(x))$$

$$\Rightarrow \sim D(x) \vee \sim L(x) \quad (2)$$

$$(\exists x)(D(x) \wedge I(x))$$

$$\Rightarrow D(A) \wedge I(A)$$

$$\Rightarrow D(A) \quad (3)$$

$$I(A) \quad (4)$$

$$\sim(\exists x)(I(x) \wedge \sim R(x))$$

$$\Rightarrow (\forall x) \sim(I(x) \wedge \sim R(x))$$

$$\Rightarrow (\forall x)(\sim I(x) \vee R(x))$$

$$\Rightarrow \sim I(x) \vee R(x) \quad (5)$$

换名后得字句集:

$$\sim R(x_1) \vee L(x_1)$$

$$\sim D(x_2) \vee \sim L(x_2)$$

$$D(A)$$

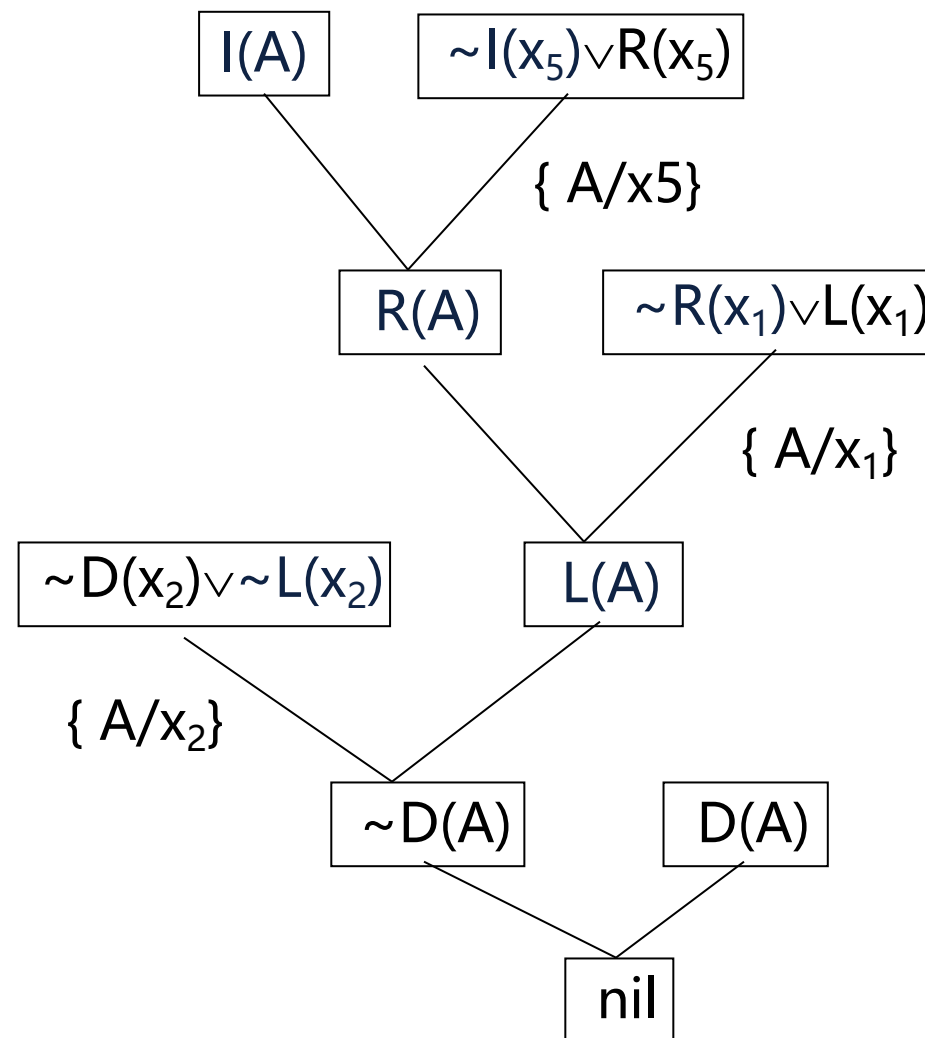
$$I(A)$$

$$\sim I(x_5) \vee R(x_5)$$

例题的归结树



$\sim R(x_1) \vee L(x_1)$
 $\sim D(x_2) \vee \sim L(x_2)$
 $D(A)$
 $I(A)$
 $\sim I(x_5) \vee R(x_5)$



>> 例子



已知: $(\forall x)[AT(John, x) \rightarrow AT(Fido, x)]$

$AT(John, School)$

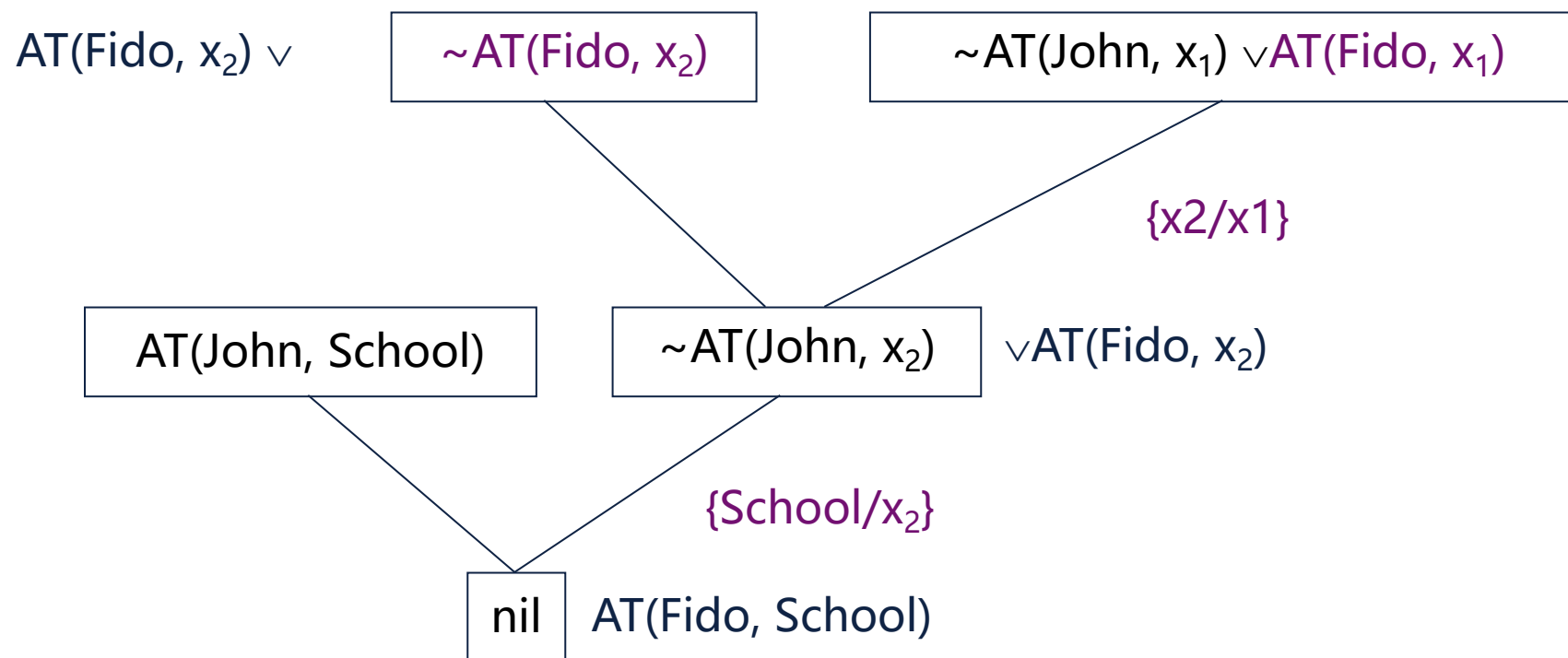
求证: $(\exists x)AT(Fido, x)$

子句集:

$\sim AT(John, x_1) \vee AT(Fido, x_1)$

$AT(John, School)$

$\sim AT(Fido, x_2)$

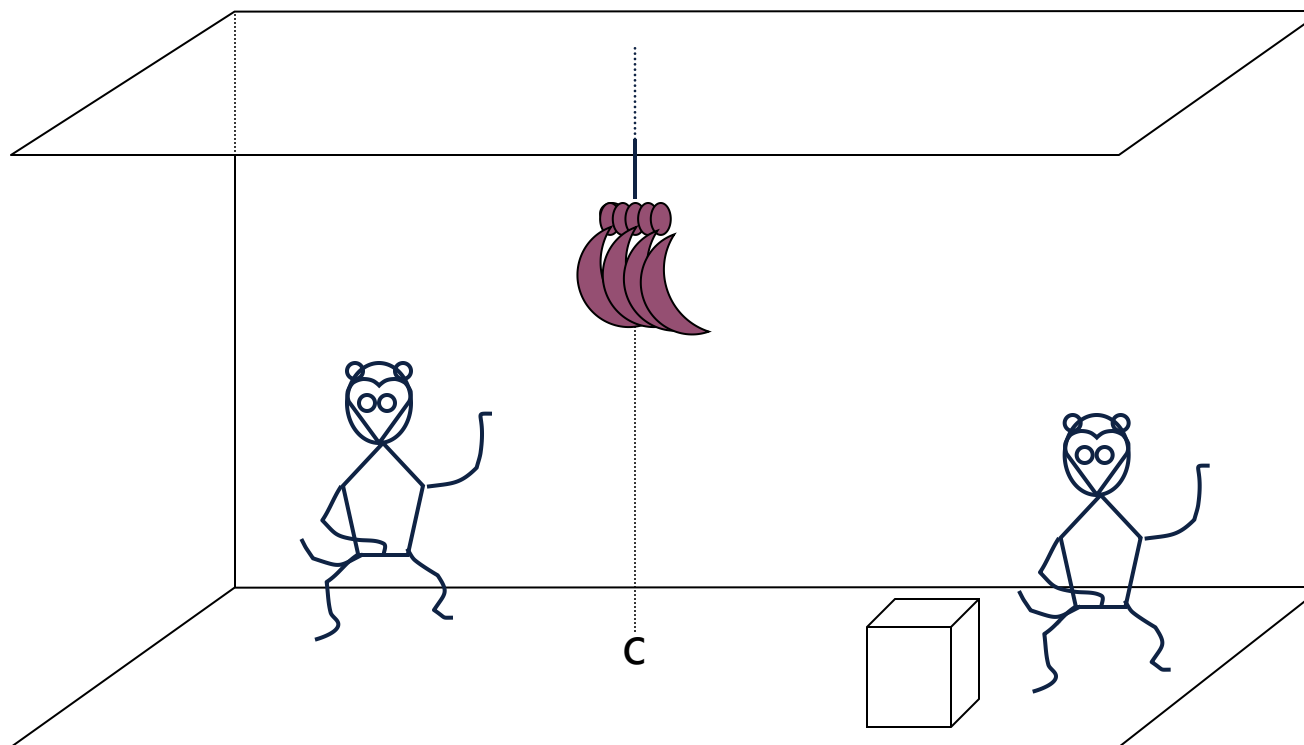


- 先进行归结，证明结论的正确性；
 - 用重言式代替结论求反得到的子句；
 - 按照证明过程，进行归结；
 - 最后，在原来为空的地方，得到的就是提取的回答。
 - 修改后的证明树称为修改证明树
-
- 还有一种办法，就是“或”上一个ANSWER (x)。寻求NIL变成寻求得到ANSWER (值)。这时候答案就是x=值
 - 理解成，只有 x=值 的时候，才推出矛盾。因为用的是反证法，这时才成立



- 这也是用推理获得一般性答案的一个方法
- 虽然现在不常用，但是很重要的
- 使得我们第一次觉得：通过机械的方法，还真能得到某种智能
- 我们学的人工智能原理，最深印象就是猴子摘到了香蕉

例：猴子摘香蕉问题



已知:

1, $\sim \text{ON}(s_0)$

2, $(\forall x)(\forall s)(\sim \text{ON}(s) \rightarrow \text{AT}(\text{box}, x, \text{push}(x, s)))$

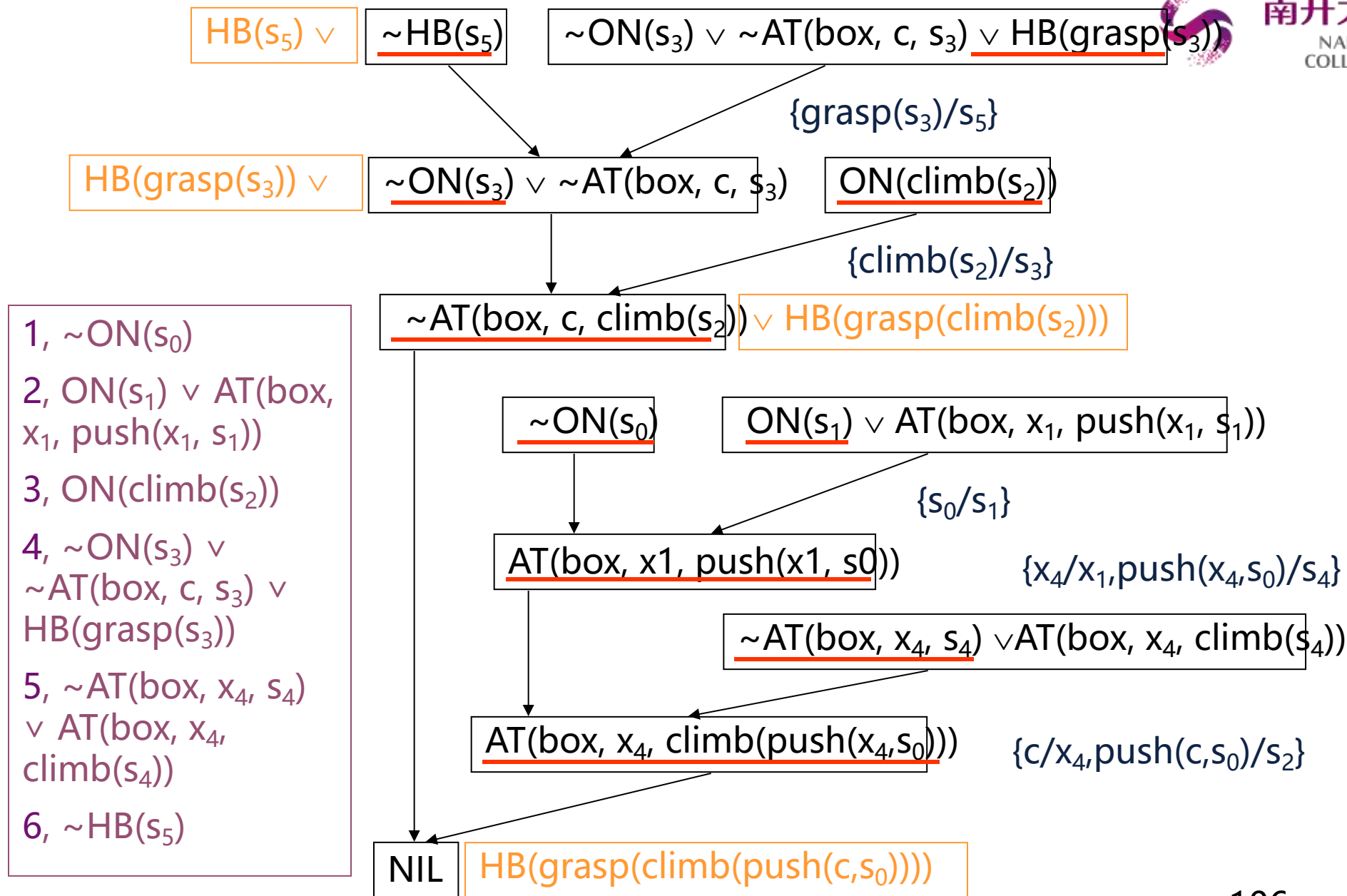
3, $(\forall s)(\text{ON}(\text{climb}(s)))$

4, $(\forall s)((\text{ON}(s) \wedge \text{AT}(\text{box}, c, s)) \rightarrow \text{HB}(\text{grasp}(s)))$

5, $(\forall x)(\forall s)(\text{AT}(\text{box}, x, s) \rightarrow \text{AT}(\text{box}, x, \text{climb}(s)))$

求解: $(\exists s)\text{HB}(s)$

- 1, $\sim \text{ON}(s_0)$
- 2, $\text{ON}(s_1) \vee \text{AT}(\text{box}, x_1, \text{push}(x_1, s_1))$
- 3, $\text{ON}(\text{climb}(s_2))$
- 4, $\sim \text{ON}(s_3) \vee \sim \text{AT}(\text{box}, c, s_3) \vee \text{HB}(\text{grasp}(s_3))$
- 5, $\sim \text{AT}(\text{box}, x_4, s_4) \vee \text{AT}(\text{box}, x_4, \text{climb}(s_4))$
- 6, $\sim \text{HB}(s_5)$





- 归结原理作为六十年代里程碑式的成果
 - 大家要了解个大概
 - 有用时再仔细读一读
-
- 基本原理与解线性方程有些类似
 - 虽然有些枯燥，但归结原理是很巧妙的
 - 记住了解线性方程，基本上就记住了归结原理



- 这两堂课，讲了传统人工智能最重要的两个技术：
- 搜索 和 归结原理
- 接下去，请姜桂飞老师讲两堂博弈论的课
- 博弈论知识非常重要，是一个新的视角！
- 再往后，是孙羽菲老师给大家讲新人工智能5次
- 穿插讲一次知识图谱和NKI的使用
- 最后我们上一堂学生实验心得的课，请同学们讲！



谢谢！ 下课